Software Architecture and Design Specification

Project: Online Internship & Placement Preparation Portal (OIPPP)

Version: 1.0

Authors: Deekshith Gowda G, Sharath H G, Guru R Bharadwaj, E S Sukruth

Date: 05-10-2026

Status: Submitted for review

Revision History

Version	Date	Author	Change Summary
0.1	29-09-2026	E S Sukruth	Component diagram, pattern choice, stack
0.2	02-10-2026	Guru R Bharadwaj	Sequence diagrams for apply and status update, API for applications
0.3	03-10-2026	Deekshith Gowda G, Sharath H G	Auth and listing APIs, security architecture
1.0	05-10-2026	All	Review comments closed

Approvals

Role	Name	Signature/Date
Architecture owner	E S Sukruth	
Security reviewer	Deekshith Gowda G	
Developer	Sharath H G	
QA lead	Guru R Bharadwaj	
Course faculty		

1. Introduction

1.1 Purpose

This document describes the architecture and the design of OIPPP release 1.0, and explains how they satisfy the requirements in the OIPPP SRS v1.0.

1.2 Scope

It covers the React client, the Express API server and its modules, the MongoDB data model, the Python analytics service and the mail integration. Deployment is described only as far as the demo setup on free hosting needs it.

1.3 Audience

The four developers, the course faculty, whoever tests the system, and anyone who maintains it after the semester.

1.4 Definitions

API: application programming interface. JWT: JSON Web Token. RBAC: role-based access control. ADR: architecture decision record. STRIDE: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege. SPA: single-page application. TLS: Transport Layer Security. Other terms are in SRS Appendix A.

2. Document Overview

2.1 How to use this document

Section 3 is the architecture: goals, components, the chosen pattern, the stack, risks, traceability and the security model. Section 4 is the design: sequence diagrams for the three most involved flows, the REST API, error handling and the UI. Diagram sources (PlantUML) are in docs/diagrams in the repository so they can be changed with the code.

2.2 Related Documents

- OIPPP Software Requirements Specification v1.0 (SRS)
- OIPPP Software Test Plan v1.0 (STP)
- Requirement Traceability Matrix (SRS Appendix C and STP section 13)
- CONTRIBUTING.md (ownership and working agreement)

3. Architecture

3.1 Goals & Constraints

Goals

- Keep each member's feature area in its own module so four people can work in parallel without stepping on each other.
- Search under 2 s and other calls under 1 s at the 95th percentile (PERF-1, PERF-2).
- No lost test answers and no duplicate applications (SAFE-1, QA-3).
- Role and ownership checks in one place so they cannot be forgotten in a new route (AUTH-6, SEC-3).

Constraints

- MERN stack plus Python, required by the problem statement.
- Free hosting only: Atlas M0 (512 MB), Render free web services that sleep after 15 minutes idle.
- One semester, four people, two-week sprints.

3.2 Stakeholders & Concerns

Stakeholder	Main concerns
Students	Finding relevant listings fast, not losing a test attempt, privacy of resume and CGPA
Recruiters	Simple posting, seeing only serious applicants, quick status updates
Placement admin	Control over who posts, accurate statistics, audit trail
Developers (team)	Clear module boundaries, testability, simple local setup
Course faculty	Traceability from requirements to code and tests, sound design choices

3.3 Component (UML) Diagram

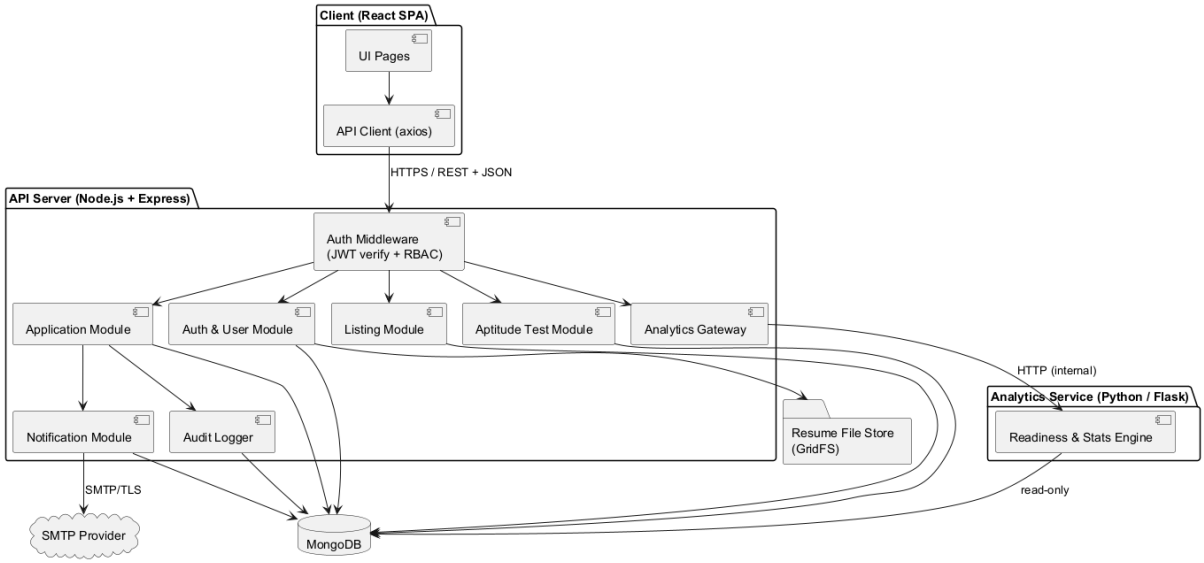


Figure 1: OIPPP component diagram

3.4 Component Descriptions

Component	Responsibility	Owner
UI Pages (React)	Screens for each role; routing with React Router; form validation before calling the API	All, per feature
API Client	axios instance that adds the token, handles 401 by sending the user to login, and turns error JSON into messages	Deekshith
Auth Middleware	Verifies the JWT, checks the account is still ACTIVE, and checks the role list attached to each route	Deekshith
Auth & User Module	Registration, login, lockout, profiles, resume upload to GridFS, recruiter approval	Deekshith
Listing Module	Listing CRUD, search with text index and filters, bookmarks, recommendation scoring, the deadline closing job	Sharath
Application Module	Apply, status state machine, applicant lists for recruiters, ownership checks	Guru
Notification Module	Stores in-app notifications, sends email through Nodemailer with retry	Guru
Audit Logger	Append-only collection of who changed what and when	Guru
Aptitude Test Module	Question bank, test builder, attempts with server-side timer, auto-save, scoring	Sukruth
Analytics Gateway	Forwards analytics requests to the Python service and caches results for 5 minutes	Sukruth

Component	Responsibility	Owner
Analytics Service (Flask)	Computes readiness scores and batch statistics with pandas over read-only queries	Sukruth
MongoDB / GridFS	All persistent data and resume files	Shared

3.5 Chosen Architecture Pattern and Rationale

We use a layered client-server architecture, and inside the API server we split the code into feature modules (a modular monolith). Each module has three layers: routes/controller, service, and Mongoose model. The client never talks to the database, and controllers never call models directly.

Why this choice:

- The modules match the ownership split in CONTRIBUTING.md, so each person mostly edits one folder.
- One deployable API keeps hosting free and simple; transactions across applications and audit log stay in one process.
- Layers make services unit testable with an in-memory MongoDB, which we need for the 70% coverage target (QA-4).

Alternatives considered:

- Microservices per feature. Rejected: four free services would each sleep separately, cross-service calls would slow every request, and the team has no time to run that.
- Serverless functions. Rejected: cold starts would break the 1 s target and the test timer needs a steady clock.
- The analytics part is the one exception, a separate Python service, because the statistics are easier in pandas and it only reads data.

ADR-1 (layered modular monolith) and ADR-2 (server-side test timer instead of client timer) are recorded in docs/adr in the repository.

3.6 Technology Stack & Data Stores

Layer	Technology	Notes
Client	React 18, Vite, React Router 6, axios, Recharts	Deployed as static site
API	Node.js 20, Express 4, Mongoose 8, Joi, jsonwebtoken, bcrypt, multer, express-rate-limit, helmet	One service on Render
Analytics	Python 3.11, Flask 3, pandas, pymongo	Read-only DB user
Data	MongoDB 7 on Atlas M0; GridFS for resumes	Indexes listed in SRS section 6
Mail	Nodemailer over SMTP (Brevo free tier)	
Testing	Jest, Supertest, mongodb-memory-server, Postman, JMeter, OWASP ZAP	
Tooling	GitHub, GitHub Actions CI, Jira (Scrum board), ESLint, Prettier	

3.7 Risks & Mitigations

Risk	Mitigation
Free Render instance sleeps; first request after idle takes about 30 s	Health-check ping every 10 minutes during demos; the client shows a 'waking server' message
Atlas M0 storage limit (512 MB) filled by resumes	2 MB resume limit, one resume per student; about 250 MB at 2,000 students
Student loses connection during a test	Server-side timer and per-answer auto-save (TST-4, SAFE-1)
Two submissions of the same application at once	Unique compound index, duplicate key mapped to 409
Email provider daily quota exceeded	Retry queue in the notifications collection; in-app notification is always written first
Team member unavailable near a deadline	Each module has a second reviewer who knows its code

3.8 Traceability to Requirements

Requirements	Feature	Component(s)	Code location
AUTH-1 to AUTH-6	Authentication and Role-Based Access	Auth & User Module, Auth Middleware	server/src/modules/auth/
PROF-1 to PROF-4	Profile and User Administration	Auth & User Module, Resume File Store	server/src/modules/users/
LST-1 to LST-4	Internship Listing Management	Listing Module	server/src/modules/listings/
SRCH-1 to SRCH-5	Search, Filtering, Bookmarks and Recommendations	Listing Module	server/src/modules/listings/
APP-1 to APP-5	Application Submission and Tracking	Application Module, Audit Logger	server/src/modules/applications/
REV-1 to REV-4	Recruiter Review and Notifications	Application Module, Notification Module	server/src/modules/applications/
TST-1 to TST-6	Aptitude Test Engine	Aptitude Test Module	server/src/modules/tests/
ANL-1 to ANL-3	Progress Tracking and Analytics	Analytics Gateway, Analytics Service	analytics/app.py
PERF-1, PERF-2	Performance	Listing Module (text + compound indexes, pagination), Analytics Gateway cache	server/src/modules/listings/
SAFE-1	No lost answers	Aptitude Test Module (auto-save, server timer)	server/src/modules/tests/
SEC-1 to SEC-6	Security	Auth Middleware, Auth & User Module, Audit Logger, helmet/rate-limit	server/src/middleware/

3.9 Security Architecture

Trust boundaries: browser to API (public internet), API to MongoDB and analytics service (private, credentials in environment variables), API to SMTP provider (TLS). Only the API holds write credentials for the database; the analytics service uses a read-only user.

Threat model (STRIDE)

Threat	Example in OIPPP	Control	Req.
Spoofing	Someone logs in as another student	bcrypt hashes, lockout after 5 failures, rate limit, JWT signed with HS256 secret, 60 min expiry	AUTH-4, AUTH-5, SEC-1, SEC-5

Threat	Example in OIPPP	Control	Req.
Tampering	Student edits a request to set their application to OFFER	Status changes only through recruiter route with ownership check and state machine; Joi validation; $-key rejection	REV-2, SEC-4
Repudiation	Recruiter denies rejecting a candidate	Append-only audit log with actor and timestamp	APP-4, SEC-6
Information disclosure	Recruiter downloads resumes of students who never applied	Resume route checks owner / applied recruiter / admin; HTTPS; no personal data in logs	SEC-2, SEC-3
Denial of service	Script floods the login endpoint	express-rate-limit, 100 KB body limit, pagination limits	SEC-5
Elevation of privilege	Student calls an admin route	Role list on every route checked by middleware; role read from DB on token refresh	AUTH-6

Secrets (JWT secret, DB URI, SMTP password) live in environment variables, never in the repository; .env is git-ignored and .env.example lists the names only. helmet sets security headers, and CORS allows only the client's origin.

4. Design

4.1 Design Overview

Every server module has the same shape: routes.js declares paths and the roles allowed; controller.js reads the request and shapes the response; service.js holds the rules; model.js is the Mongoose schema. Services throw typed errors (AppError with a code and HTTP status) and one error middleware turns them into JSON. On the client, each feature has a pages folder and an api file.

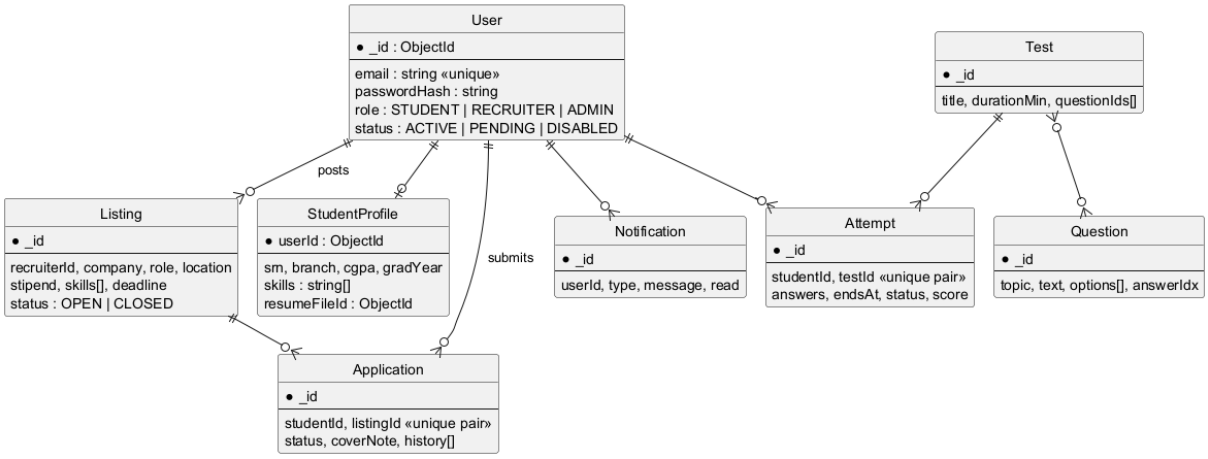


Figure 2: Data model

4.2 UML Sequence Diagrams

4.2.1 Student applies to an internship (APP-1, APP-2, APP-4)

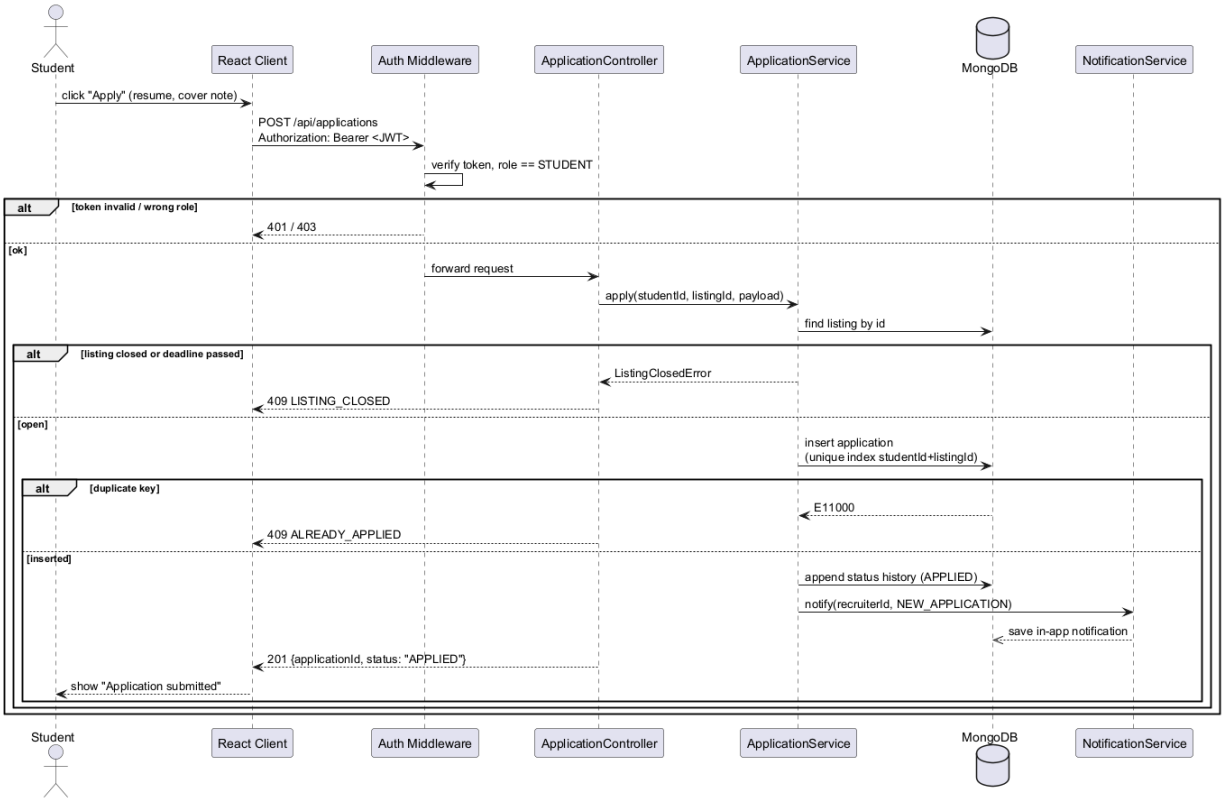


Figure 3: Apply to internship

4.2.2 Timed test attempt with auto-save (TST-3 to TST-6, SAFE-1)

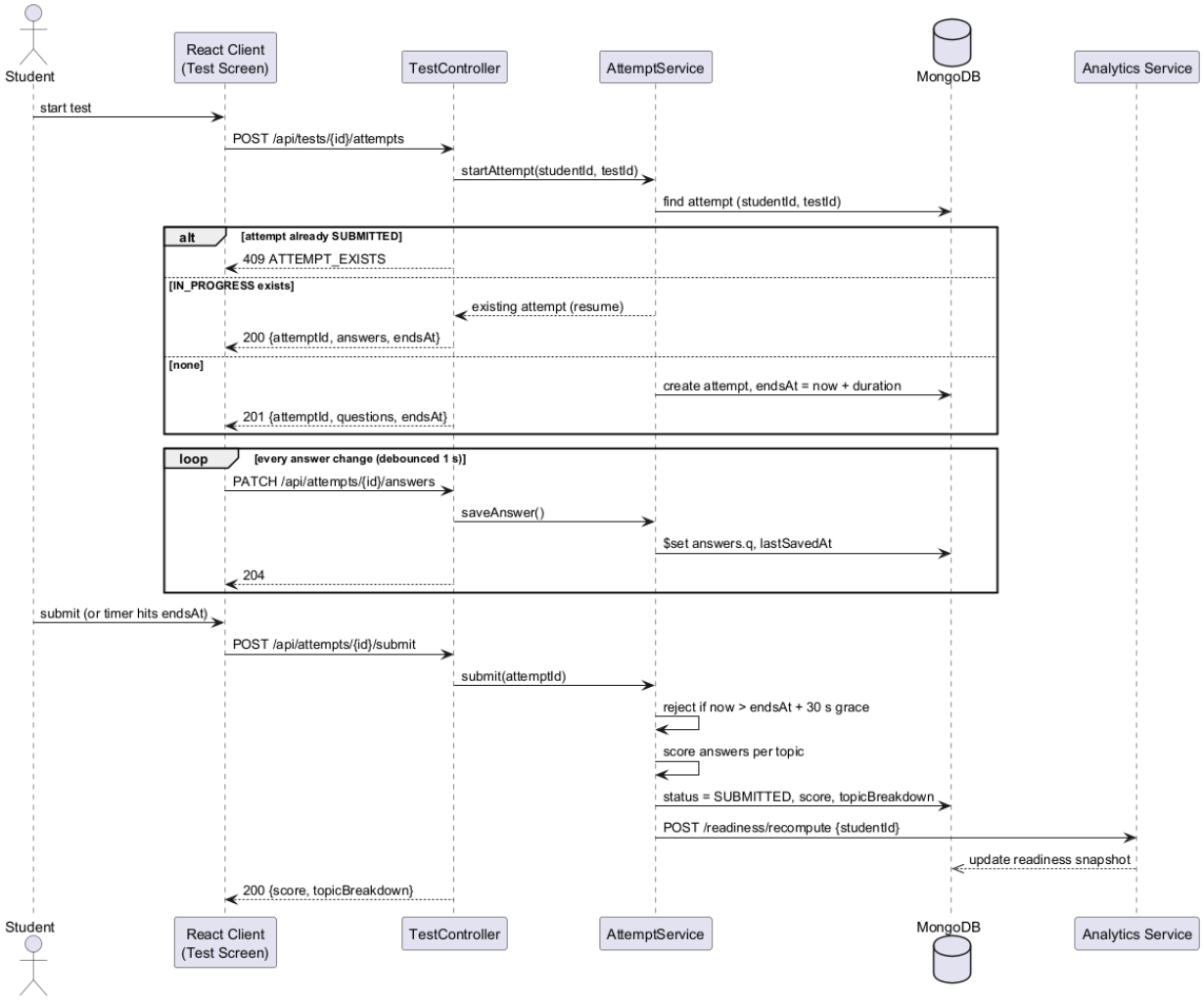


Figure 4: Test attempt

4.2.3 Recruiter updates application status (REV-2 to REV-4)

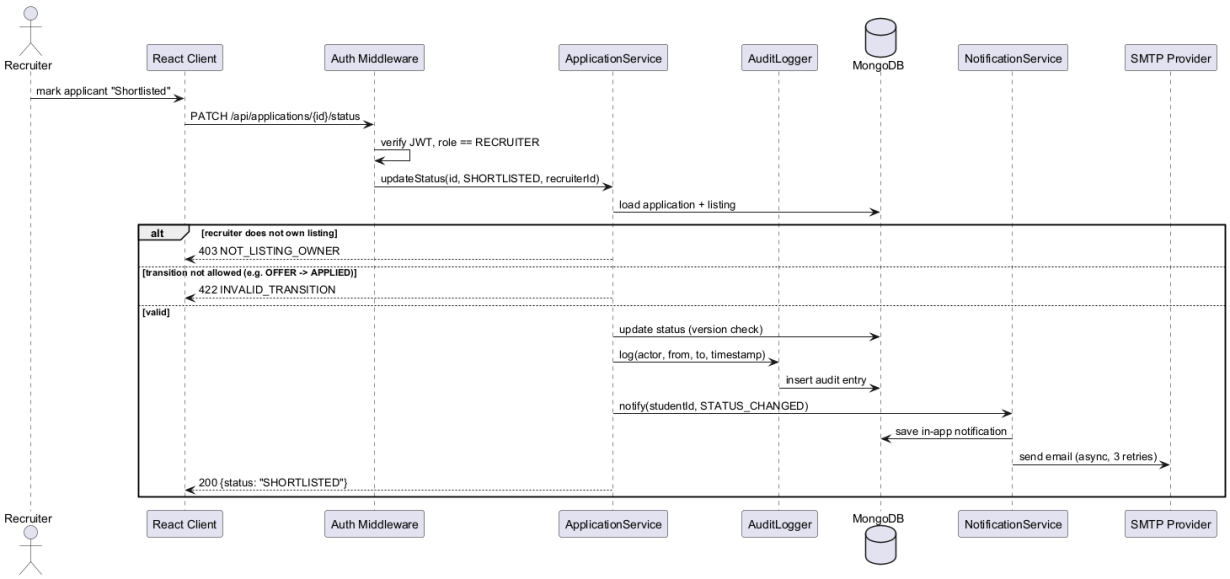


Figure 5: Status update and notification

4.3 API Design

Base path /api, JSON in and out, token in Authorization: Bearer <token>. Errors always have the shape {"error": {"code": "STRING_CODE", "message": "text for the user", "details": [...]}}. Lists

take page and limit (max 50).

4.3.1 Auth and users (Deekshith)

Method	Endpoint	Role	Request	Response	Errors
POST	/auth/register	Public	{name, email, password, role, srn?}	201 {user}	400 VALIDATION, 409 EMAIL_TAKEN
POST	/auth/login	Public	{email, password}	200 {token, user}	401 BAD_CREDENTIALS, 423 LOCKED, 429
GET	/users/me	Any	-	200 {user, profile}	401
PUT	/users/me/profile	Student	{branch, cgpa, gradYear, skills[]}	200 {profile}	400
POST	/users/me/resume	Student	multipart file (PDF <= 2 MB)	200 {resumeFileId}	400 BAD_FILE, 413
GET	/users/:id/resume	Owner / applied recruiter / admin	-	200 application/pdf	403, 404
PATCH	/admin/recruiters/:id	Admin	{decision: APPROVE|REJECT, reasonCode?}	200	404, 422

4.3.2 Listings (Sharath)

Method	Endpoint	Role	Request	Response	Errors
POST	/listings	Recruiter (approved)	{company, title, location, mode, stipend, skills[], minCgpa?, deadline}	201 {listing}	400, 403 NOT_APPROVED
GET	/listings	Student	?q&location&mode&minStipend&skill&sort&page	200 {items[], total, page}	400
PATCH	/listings/:id	Owning recruiter	partial listing or {status: CLOSED}	200	403 NOT_OWNER, 404
POST/DELETE	/listings/:id/bookmark	Student	-	204	404, 422 LIMIT
GET	/listings/feed	Student	-	200 {items[10]}	-

4.3.3 Applications (Guru)

Method	Endpoint	Role	Request	Response	Errors
POST	/applications	Student	{listingId, coverNote?}	201 {applicationId, status}	409 LISTING_CLOSED, 409 ALREADY_APPLIED, 422 NOT_ELIGIBLE
GET	/applications/mine	Student	-	200 {items[]}	-
GET	/listings/:id/applications	Owning recruiter	?status&page	200 {items[]}	403

Method	Endpoint	Role	Request	Response	Errors
PATCH	/applications/:id/status	Owning recruiter	{status, reasonCode?}	200 {status}	403 NOT_LISTING_OWNER, 422 INVALID_TRANSITION,400 REASON_REQUIRED
GET	/notifications	Any	?unread	200 {items[]}	-

4.3.4 Aptitude tests (Sukruth)

Method	Endpoint	Role	Request	Response	Errors
POST	/questions	Admin	{topic, text, options[], answerIdx, difficulty}	201	400
POST	/tests	Admin	{title, durationMin, questionIds[]}	201	400
POST	/tests/:id/attempts	Student	-	201 new / 200 resume {attemptId, questions, answers, endsAt}	409 ATTEMPT_EXISTS
PATCH	/attempts/:id/answers	Student (owner)	{questionId, optionIdx}	204	409 ATTEMPT_CLOSED
POST	/attempts/:id/submit	Student (owner)	-	200 {score, topicBreakdown}	409

4.3.5 Analytics (Sukruth)

Method	Endpoint	Role	Request	Response	Errors
GET	/analytics/me	Student	-	200 {readiness, trend[], weakTopics[], appCounts}	-
GET	/analytics/placement	Admin	?branch&gradYear	200 {placed, total, byCompany[], ...}	-
GET	/analytics/placement.csv	Admin	?branch&gradYear	200 text/csv	-

4.4 Error Handling, Logging & Monitoring

- All service errors are AppError(code, status, message). A single Express error middleware returns the standard JSON shape. Unknown errors return 500 INTERNAL with a request id and no stack trace.
- Validation errors (Joi) return 400 with one entry per field so the form can show messages under each field.
- Duplicate key errors from MongoDB (E11000) are mapped to 409 with the right code.
- Email sending failures do not fail the request. The notification is kept with status PENDING and retried up to 3 times with 1, 5 and 15 minute gaps.
- Logs use pino in JSON with request id, route, status and time taken. Passwords, tokens, resumes and CGPA are never logged.

- Monitoring for the demo: /health endpoint, UptimeRobot free check every 5 minutes, and a daily count of 5xx responses and failed emails from the logs.
- The client shows a banner for 5xx and network errors, keeps form input, and lets the user retry.

4.5 UX Design

- Role-based navigation: each role only sees its own menu items.
- Status chips with fixed colours (Applied grey, Shortlisted blue, Interview purple, Offer green, Rejected red) plus a text label, so colour is not the only signal.
- Test screen: countdown always visible, question palette shows answered and unanswered, small 'Saved' tick after each answer, warning at 5 minutes left.
- Forms validate on blur and keep input after a server error. Keyboard navigation works on all forms; contrast ratio at least 4.5:1.
- Mobile layout from 360 px; filters collapse into a drawer.

4.6 Open Issues & Next Steps

- Recommendation is plain skill overlap for 1.0; weighting by test topic scores is planned for a later release.
- Decide whether recruiters can download all resumes of a listing as a zip.
- Refresh tokens are not in 1.0; users log in again after 60 minutes.
- Load test on the actual Render instance in Sprint 3.

5. Appendices

5.1 Glossary: see SRS Appendix A, plus ADR, STRIDE, GridFS.

5.2 References: ISO/IEC/IEEE 42010:2011; OWASP Top 10 2021; OWASP ASVS 4.0; NIST SP 800-160 Vol. 1; OIPPP SRS v1.0.

5.3 Tools: PlantUML (all diagrams), draw.io, Postman, Jira, GitHub.